

1.Metadata 생성과 검색

Tool 입력값 재사용 설계

Metadata 개념: 검색 가능한 지식 단위의 설계

Chunk와 Metadata의 관계

Chunk는 긴 기술 문서를 RAG 검색 대상으로 분할한 지식 단위입니다. Metadata는 각 chunk를 설명하는 구조화 정보로, 검색·필터링·근거 추적·Tool 연계의 기준이 됩니다.

→ 검색 필터링 기준

알람 코드, 설비 ID, 공정명 등으로 관련 chunk를 빠르게 좁힘

→ 근거 추적 구조

검색 결과가 어떤 문서의 어떤 위치에서 왔는지 추적 가능

→ Tool 연계 입력값

3일차 search_manual MCP Tool의 입력·출력 구조 설계로 연결

Metadata 예시 (교육용 가상 데이터)

alarm_code: ALM-TEMP-402

equipment_id: EQP-EV-03

equipment_type: 증착 설비

symptom: 온도 편차

quality_metric: 품질 영향 확인

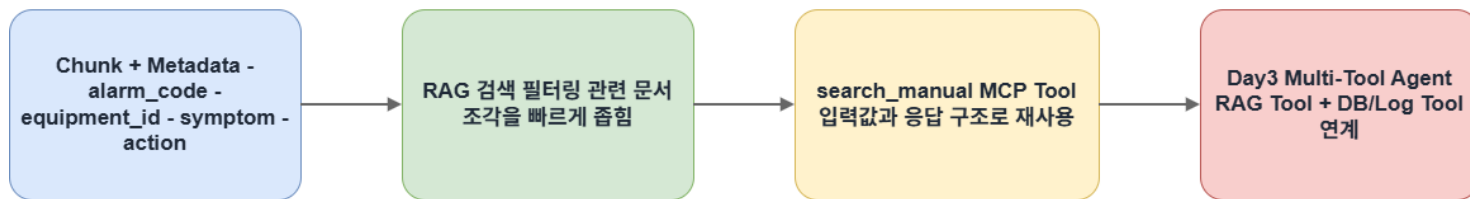
action: 1차 확인·조치 항목

keywords: 검색 recall 보완

 Metadata는 단순한 이름표가 아닙니다. 검색·필터링·근거 추적·Tool 연계를 위한 구조화 설계 요소입니다. 2일차 검색 품질과 3일차 search_manual Tool 입출력 구조 설계에 직접 연결됩니다.

Metadata의 Day3 연결

2일차 검색 필터링 기준이 3일차 search_manual MCP Tool 입출력 구조로 확장됩니다.



발표 핵심: Metadata는 문서 검색용 보조 정보에 그치지 않고, 다음 단계의 Tool Contract 설계 기준으로 이어집니다.

chunk_builder.py가 추출하는 주요 Metadata 항목

ZIP의 chunk_builder.py는 각 문서 조각에 아래 metadata를 구조화하여 추출합니다. 각 항목은 검색 조건, 답변 근거, Tool 입력값으로 재사용되는 설계 요소입니다.

Metadata 항목	의미	설계 관점
alarm_code	알람 코드	알람 매뉴얼 검색 및 DB/Log Tool 호출 조건
equipment_id	설비 ID	특정 설비 로그·상태 조회 연결
equipment_type	설비 종류	유사 설비 유형별 검색 범위 설정
process_name	공정 이름	공정별 문서·품질 기준 검색 조건
symptom	증상	알람 코드 부재 시 증상 기반 검색 활용
quality_metric	품질 지표	품질 영향 여부 판단 근거
action	조치 방향	답변의 1차 확인·조치 항목 구성
keywords	주요 키워드	검색 recall 보완 및 결과 해석 보조



Metadata는 문서 조각을 설명하는 정보이면서, 이후 검색 조건·답변 근거·Tool 입력값으로 재사용될 수 있는 설계 요소입니다.

Metadata 예시: 질문과 문서를 연결하는 기준

교육용 제조 시나리오에서 metadata는 질문과 문서 조각을 연결하는 중간 기준입니다. 각 항목이 어떤 검색·Tool 호출 조건으로 작동하는지 확인합니다.

ALM-TEMP-402

알람 코드 기반 검색의 기준 — 알람 매뉴얼 chunk를 빠르게 식별

EQP-EV-03

특정 설비 단위의 DB/Log Tool 호출 조건으로 연결 가능

증착 설비

설비 유형별 검색 범위를 정하는 기준 — 유사 설비 문서 포함

온도 편차

알람 코드가 명확하지 않을 때 증상 기반 검색에 활용

품질 영향 확인

답변에서 품질 기준서·품질 지표를 함께 확인해야 한다는 신호

 Metadata는 단순 설명 정보가 아닙니다. 질문 → 문서 검색 → DB/Log Tool 호출 → 답변 근거를 연결하는 중간 기준입니다.

Metadata 품질이 검색 품질에 미치는 영향

Metadata가 잘 설계된 경우

정확한 필터링

알람 코드·설비 유형·공정명이 검색 조건과 일치하여 관련 chunk를 정확하게 식별

근거 추적 가능

doc_name, chunk_id, section_title이 명확하여 답변 근거 위치 추적 가능

Tool 입력값 재사용

3일차 search_manual MCP Tool의 입력 구조로 직접 연결

Metadata가 누락·오추출된 경우

검색 범위 확산

관련 없는 문서가 근거로 사용되거나, 관련 문서가 검색되지 않음

근거 추적 불가

출처가 불명확하여 Trace 기반 실행 검토가 어려워짐

LLM 답변 품질 저하

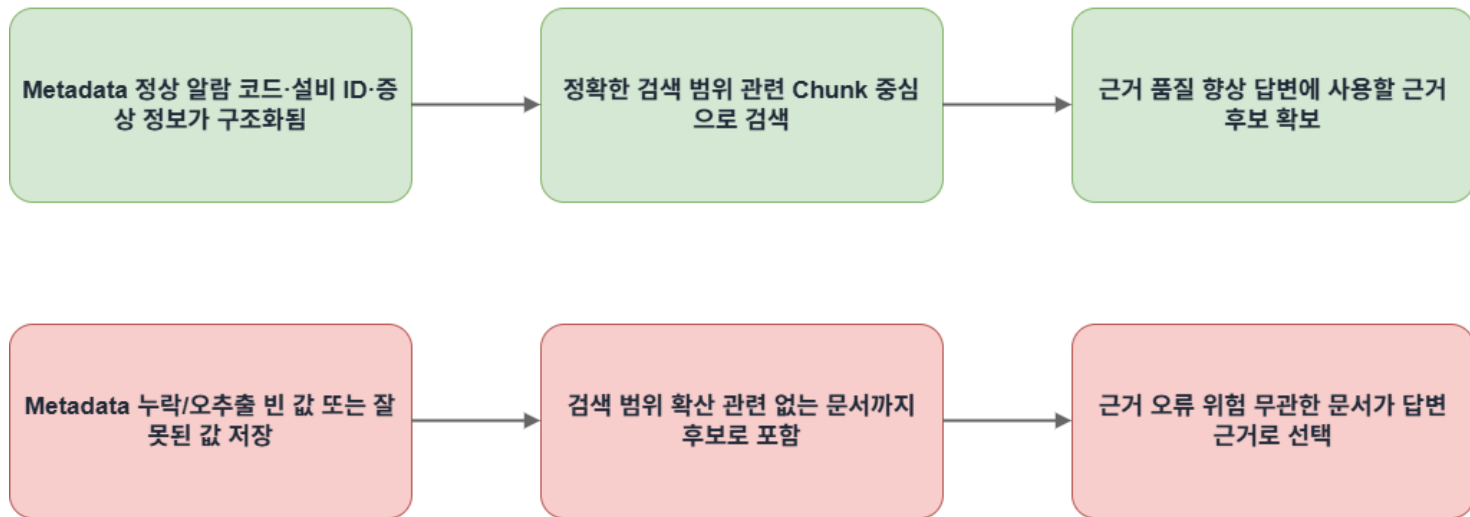
앞단 검색 품질이 낮으면 이후 LLM 답변도 개선되기 어려움



Metadata 품질은 RAG 답변 품질의 앞단 기준입니다. 검색 품질 → 답변 근거 정확성 → 3일차 Tool 입력값 설계에 직접 영향을 줍니다.

Metadata 누락의 영향

Metadata 품질은 검색 품질의 앞단 기준입니다.



chunk_preview.json에서 Metadata 확인하는 방법

파일 위치 및 역할

outputs/day2/chunk_preview.json

이후 검색 로직이 재사용하는 구조화된 chunk 데이터입니다. 각 chunk별로 metadata가 올바르게 생성되었는지 확인하는 기준 자료입니다.

확인 항목

- chunk_id — 문서 조각 고유 식별자
- doc_name — 원본 문서명
- alarm_code — 알람 코드 추출 여부
- equipment_id — 설비 ID 추출 여부
- symptom — 증상 키워드 추출 여부
- action — 조치 항목 추출 여부
- keywords — 검색 보완 키워드

확인 관점

값을 임의로 암기하는 것이 아니라, 실제 파일에서 어떤 metadata가 붙었고 어떤 값이 비어 있는지 확인하는 것이 중요합니다.

01

chunk_preview.json 열기

outputs/day2 폴더에서 파일 확인

02

각 chunk별 metadata 점검

alarm_code, equipment_id 등 핵심 항목 추출 여부 확인

03

빈 값 식별

누락된 metadata 항목이 검색 품질에 미치는 영향 검토

04

이후 연결 확인

rag_test_results.md, langgraph_rag_trace.md, 3일차 Tool 응답 구조와의 연결 점검



chunk_preview.json은 chunk와 metadata를 확인하는 미리보기 파일이면서, 검색 품질과 근거 추적 가능성을 점검하는 기준 자료입니다.

다음 실습 연결: Metadata → RAG 검색

문서가 chunk로 분할되고 각 chunk에 metadata가 구조화되었습니다. 이제 사용자 질문과 관련 있는 문서 조각을 실제로 검색하는 단계로 진입합니다.

01

Chunk 구성

Metadata와 함께 구조화 완료

02

RAG 검색

rag_search.py로 관련 문서 검색

03

근거 확보

Top-3 후보 문서 선택

Metadata의 검색 시 역할

- 검색 범위를 좁히는 필터링 조건으로 작동
- 검색 결과가 어떤 문서·근거에서 나온 것인지 해석 기준 제공
- ALM-TEMP-402 반복 발생 질문 → 관련 알람 매뉴얼 chunk 식별



8장 연결 포인트

- src/day2/rag_search.py로 metadata 기반 검색 실행
- 검색 결과는 Top-3 근거 후보 형태로 정리
- 이후 State에 저장 → 3일차 search_manual MCP Tool 출력 구조로 확장

2.Top-3 RAG 검색 실행과 결과 해석

Top-3 검색 개념: 근거 후보 확보 전략

왜 Top-3인가?

Top-3 검색은 사용자 질문과 가장 관련성이 높은 근거 후보 3개를 확보하는 과정입니다. 하나의 문서 조각만 보고 원인을 단정하지 않고, 여러 근거 후보를 비교하여 답변의 신뢰도를 높입니다.

Top-3 ≠ 정답 3개

답변 생성 전에 검토할 근거 후보
집합입니다

비교 검토 가능

여러 근거 후보를 비교하여 답변
신뢰도 향상

LLM 답변 전 단계

LLM이 답변하기 전에 사용할 근거 후보를 찾고 검토하는 과정

제조 장애 대응에서의 의미

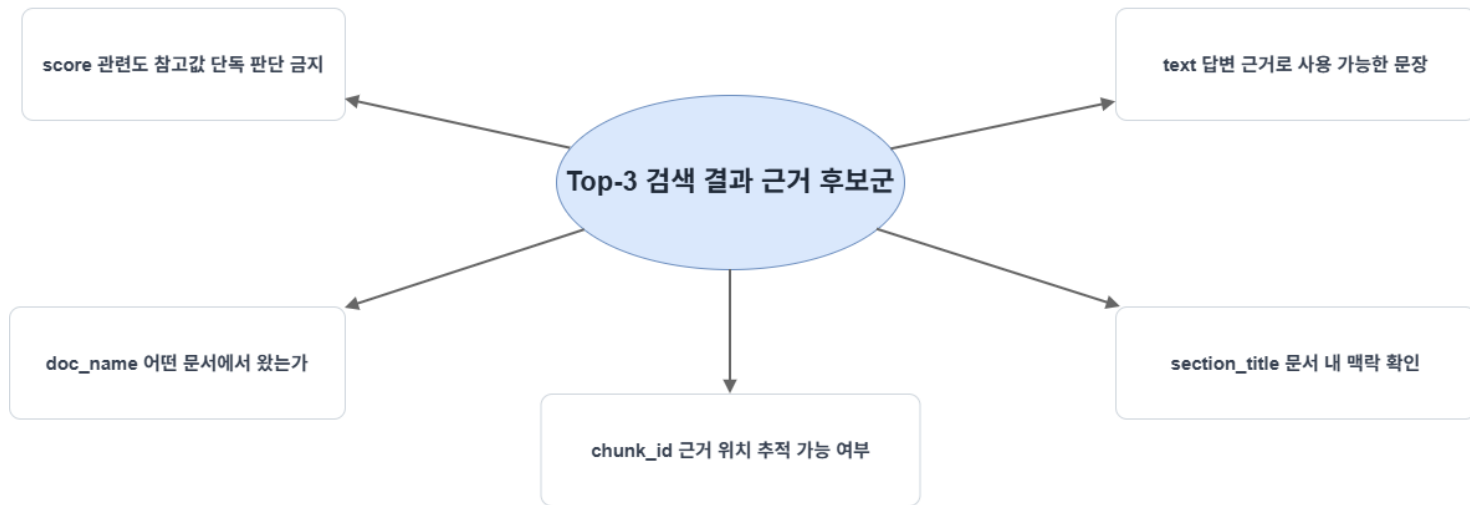
단일 문서 조각으로 원인을 단정하는 것은 위험합니다. Top-3
근거 후보를 확보함으로써:

- 반복 발생 여부 확인 가능
- 온도 편차 정도 비교 가능
- 품질 영향 근거 교차 검토 가능
- 최근 정비 이력 연계 가능

검색 품질이 낮으면 이후 LLM 답변도 개선되기 어렵습니다.

Top-3 해석 기준

Top-3는 정답 3개가 아니라 답변 전 검토할 근거 후보군입니다.



발표 핵심: 점수가 높은 1개 결과보다, Top-3 안에 답변에 쓸 수 있는 신뢰 가능한 근거 후보가 포함되는지가 중요합니다.

rag_search.py: RAG 검색 실행 파일의 역할

입력

사용자 질문 또는 sample_rag_queries.json의 표준 테스트 질의

처리

metadata 기반 필터링으로 관련 문서 chunk 검색 → Top-3 근거 후보 정리

출력

outputs/day2/rag_test_results.md에 검색 결과 저장

연결

검색된 문서 조각 → State 저장 → 답변 생성 근거로 사용

 rag_search.py는 LLM이 답변하기 전에 사용할 근거 후보를 찾는 파일입니다. 검색 결과가 부정확하면 이후 LLM 답변도 개선되기 어렵습니다. RAG 검색은 답변 품질의 앞단 품질을 결정합니다.

입력 파일: sample_rag_queries.json

파일 역할

data/sample_rag_queries.json은 RAG 검색 품질 확인용 표준 테스트 질의 목록입니다. 수강생이 직접 질문을 작성하기 전에 사용할 수 있는 기준 입력 사례입니다.

포함 질문 유형

- 알람 코드 중심 질문

ALM-TEMP-402의 의미와 1차 확인 항목

- 증상 중심 질문

온도 편차 반복 발생 시 대응 방법

- 품질 영향 질문

해당 알람이 품질 지표에 미치는 영향

설계 관점

동일한 테스트 질의를 반복 사용하면 검색 결과 변화를 재현 가능하게 확인할 수 있습니다.

- 검색 조건 변경 전후 비교 가능
- metadata 개선 효과 측정 가능
- 검색 품질 기준선 확보
- 4일차 실행 품질 평가 기반으로 연결



sample_rag_queries.json은 RAG 검색 품질을 재현 가능하게 확인하기 위한 기준 질문 목록입니다.

PowerShell 실행: rag_search.py

실행 위치 및 명령어

명령어는 manufacturing_agent_project 폴더의 루트 위치(src 폴더가 보이는 위치)에서 실행합니다.

```
uv run python src/day2/rag_search.py
```

실행 결과

- 샘플 질문 기준으로 관련 문서 chunk 검색
- Top-3 검색 결과 정리
- outputs/day2/rag_test_results.md 생성

운영 관점: Saved Result Review



Chroma/Ollama 관련 오류 발생 시 오류 해결에 오래 머무르지 않습니다.

실행 환경 문제가 있을 경우, 기존 산출물인 outputs/day2/rag_test_results.md를 기준으로 Saved Result Review 방식으로 결과 해석 실습을 진행합니다.



Vector DB 구축(Chroma 인덱스 생성)은 chroma_index_builder.py 선택 실습 또는 강사 데모로 분리됩니다. 오늘의 중심은 검색 결과 해석과 근거 후보 검토입니다.

예상 검색 결과 구성 및 검토 관점

검색 결과는 점수만 보는 것이 아니라 문서 출처, 문맥, 실제 근거 문장을 함께 검토해야 합니다.

결과 항목	의미	검토 관점
query	검색 질의	어떤 조건으로 검색했는지 확인
doc_name	원본 문서명	어떤 문서를 근거로 사용했는지 확인
chunk_id	문서 조각 ID	근거 위치 추적 가능 여부
score	관련도 점수	참고값 — 단독 판단 기준으로 사용 금지
section_title	문서 내 섹션	문서 맥락 확인
text	검색된 문서 내용	실제 답변 근거로 사용 가능한지 검토

 score가 높아도 정답으로 단정하면 안 됩니다. doc_name, section_title, 관련 문단을 반드시 함께 확인해야 합니다.

산출물 확인: rag_test_results.md

파일 위치 및 성격

outputs/day2/rag_test_results.md

Top-3 검색 결과를 사람이 읽기 좋게 정리한 Markdown 산출물입니다. 단순 저장 파일이 아니라 검색 품질 점검 자료입니다.

- 질문별 근거 후보가 적절한지 검토
- 어떤 질문으로 어떤 문서 조각이 검색되었는지 확인
- 환경 문제 시 Saved Result Review 방식으로 활용

검색 품질 점검 기준

관련성

질문의 알람 코드:증상이
문서에 명확히 포함되어
있는가

근거성

답변에 사용할 수 있는 조
치 문장이나 확인 기준이
포함되어 있는가

출처 확인

doc_name, chunk_id,
section_title이 명확한가

추적 가능성

Trace에서 어떤 근거를 사
용했는지 다시 확인할 수
있는가

검색 결과 검토 기준

rag_test_results.md는 검색 품질을 검토하는 기준 자료입니다.

관련성 질문과 문서 내용이 실제로 맞는가?

근거성 답변에 사용할 수 있는 확인·조치 문장이 있는가?

출처 확인 가능성 doc_name, chunk_id, section_title이 명확한가?

Trace 재확인 가능성 나중에 사용 근거를 다시 추적할 수 있는가?

검토 결론: 검색 결과는 점수가 아니라 “답변 근거로 쓸 수 있는가”와 “나중에 다시 추적할 수 있는가”를 기준으로 판단합니다.

Vector DB와 Embedding

rag_search.py의 기술 구성

Chroma Vector DB

검색 가능한 문서 벡터를 저장하고, 질의와 가까운 문서 조각을 찾는 역할
— 선택/확장 영역

Ollama + Embedding

문장의 의미를 숫자로 변환하여 검색을 보조 — 선택/확장 영역

chroma_index_builder.py

Chroma 인덱스 생성은 선택 실습 또는 강사 데모로 분리

2일차 기본 교재의 중심

Chroma 구축이나 Ollama 설정이 아닙니다.

핵심은 Top-3 검색 결과가 답변 근거로 쓰이는 과정입니다.

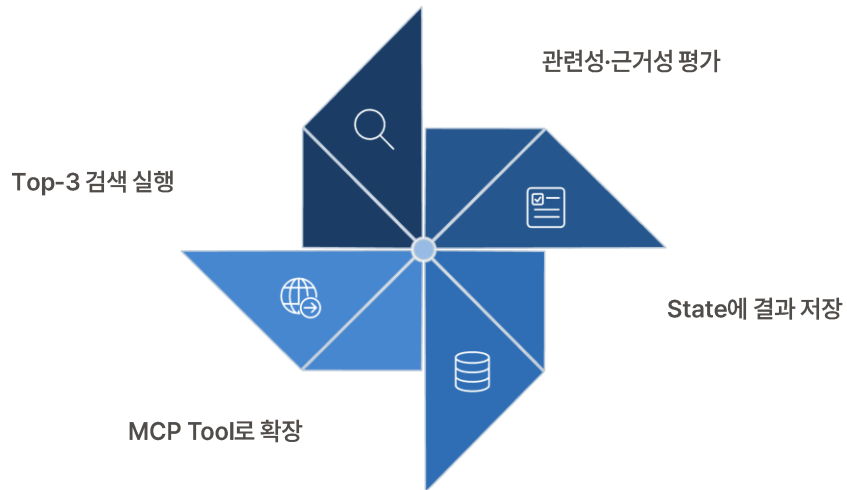
- 검색 결과 해석
- 근거 후보 검토
- State 저장 구조 이해
- Trace 기반 실행 검토



오늘은 Vector DB 구축보다 검색 결과 해석과
근거 후보 검토가 우선입니다.

다음 실습 연결: 검색 결과 평가와 State 저장

Top-3 검색을 수행했다고 해서 검색 결과가 항상 완벽한 것은 아닙니다. 검색 결과가 질문과 얼마나 관련 있는지, 답변 근거로 충분한지 반드시 확인해야 합니다.



검색 결과 평가 기준

- score만 보지 말고 문서명·섹션 제목·관련 문단을 함께 확인
- 검색 결과가 부족하면 질문을 재작성하거나 metadata 활용하여 검색 범위 조정

9장 연결 포인트

- 검색 관련성·근거성 평가 방법론
- 검색 결과를 State에 저장하는 구조
- 답변 생성과 근거 검증 흐름으로 연결

3. 검색 관련성·근거성 개선과 설계 기준

검색 품질 비교: 높은 결과 vs 낮은 결과

구분	검색 품질이 높은 결과	검색 품질이 낮은 결과	확인 포인트
관련성	질문의 알람 코드와 증상이 문서에 명확히 포함됨	다른 알람이나 일반 설명이 검색됨	질문과 문서 내용이 실제로 맞는가
근거성	답변에 사용할 수 있는 조치 문장이 나 확인 기준 포함	근거 문장이 모호하거나 답변에 직접 쓰기 어려움	답변 근거로 사용할 수 있는가
출처 확인	doc_name, chunk_id, section_title이 명확함	출처가 애매하거나 관련 위치 추적 어려움	어느 문서의 어느 부분인가
추적 가능성	검색 결과를 Trace에서 다시 확인 가능	나중에 어떤 근거를 사용했는지 확인 어려움	실행 후 근거 위치를 다시 검토할 수 있는가



score가 높다고 바로 정답으로 단정하지 않습니다. 문서명, chunk_id, 섹션 제목, 문단 내용을 반드시 함께 확인해야 합니다.



검색 품질이 높은 결과는 질문과 잘 맞고, 답변 근거로 사용할 수 있으며, Trace에서 다시 확인 가능한 결과입니다.

알람 코드만 검색한 경우의 한계

알람 코드 검색의 강점

ALM-TEMP-402처럼 알람 코드가 명확하면 관련 문서를 빠르게 찾을 수 있습니다.

검색 범위 신속 축소

알람 코드는 검색 범위를 빠르게 좁히는 강한 조건

알람 의미 파악

알람 매뉴얼에서 해당 알람의 의미, 심각도, 1차 확인 항목 식별에 유용

알람 코드만으로는 부족한 정보

- 반복 발생 여부
- 온도 편차 정도
- 품질 영향 수준
- 최근 정비 이력

같은 알람이라도 발생 상황에 따라 확인해야 할 근거가 달라질 수 있습니다.



알람 코드 검색은 좋은 출발점이지만, 그것만으로 충분하다고 단정하면 안 됩니다. 증상·공정·품질 영향 같은 상황 정보를 함께 고려해야 합니다.

증상과 상황을 함께 넣은 검색 전략

검색 방식	검색어 예시	기대 효과	한계
알람 코드만 검색	ALM-TEMP-402	알람 의미를 빠르게 찾을 수 있음	반복 발생, 품질 영향까지는 약할 수 있음
증상·상황 함께 검색	ALM-TEMP-402 반복 발생, 온도 편차, 품질 영향	실제 장애 상황에 가까운 근거를 찾을 가능성이 높음	검색어가 너무 길거나 모호하면 결과가 흩어질 수 있음

설계 원칙

좋은 검색 질의는 정확한 식별 정보와 업무 상황 정보를 균형 있게 포함해야 합니다.

- 알람 코드처럼 정확한 조건 → 검색 범위 축소
- 증상·품질 영향처럼 상황 정보 → 실제 장애 상황 반영

검색 질의 설계 시 주의사항

- 검색 질의가 너무 길거나 모호하면 검색 결과가 흩어질 수 있음
- 핵심 식별자(알람 코드)와 상황 키워드(증상, 품질 영향)의 균형 유지
- 3일차 search_manual Tool 입력 구조 설계 시 동일 원칙 적용

Top-3 안에 신뢰 가능한 근거 후보가 포함되는지 확인

Top-3 ≠ 시험 점수

가장 높은 점수 하나만 보지 말고, Top-3 안에 답변 생성에 사용할 수 있는 신뢰 가능한 근거 후보가 포함되어 있는지 확인

score는 참고값

제조 장애 대응에서는 점수보다 doc_name, chunk_id, section_title, text가 더 중요할 수 있음

출처와 근거 문장 확인

어떤 문서에서 왔고, 어떤 문장이 근거인지 함께 확인해야 함

근거 후보군 안정성

검색 품질의 핵심은 최고 점수 1개가 아니라, 답변 생성에 필요한 근거 후보군이 안정적으로 확보되는지 여부

✔ Top-3 안에 답변 근거로 사용할 수 있는 후보가 안정적으로 포함되어 있는지가 핵심입니다.

검색 품질 검토: rag_test_results.md 확인 항목

확인 항목	의미	수강생 확인 질문
query	검색에 사용한 질문	질문이 너무 짧거나 모호하지 않은가?
Top-3 결과	관련 문서 후보 3개	근거로 쓸 문서가 포함되어 있는가?
doc_name	원본 문서명	어떤 문서에서 나온 근거인가?
chunk_id	문서 조각 ID	근거 위치를 다시 추적할 수 있는가?
score	관련도 참고 점수	점수만 보고 단정하지 않았는가?
text	검색된 문단	답변에 직접 인용하거나 요약할 수 있는가?

실습 방법

outputs/day2/rag_test_results.md를 열어 query별 Top-3 검색 결과를 확인합니다. 환경 문제가 있으면 기존 산출물을 사용해 Saved Result Review 방식으로 해석 실습을 진행합니다.

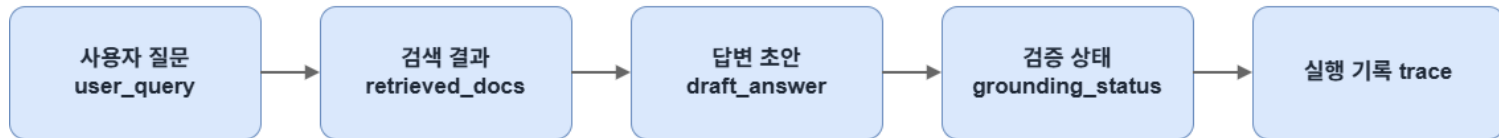
이 파일의 설계적 의미

단순 저장 파일이 아닙니다. 질문별 검색 관련성·근거성·출처 확인 가능성을 점검하는 검색 품질 검토 자료이며, 이후 State 저장과 근거 검증 흐름으로 넘길 수 있는지 판단하는 기준입니다.

4. State 설계와 Node 간 전달 구조

Memory와 State 연결

여기서 Memory는 장기 기억보다 실행 중 상태 유지 구조에 가깝습니다.



State = Agent 실행 중 공유되는 업무 처리 문맥

발표 핵심: State는 Node 사이를 이동하면서 질문, 근거, 답변, 검증 상태, Trace를 이어 주는 실행 중 Memory입니다.

State 개념: Node 사이에서 공유되는 업무 처리 문맥

Agent는 한 번에 모든 일을 끝내지 않습니다

RAG Agent는 여러 단계를 순서대로 실행합니다. 이 과정에서 중간 정보를 일관되게 저장하고 다음 단계로 전달할 구조가 필요합니다.



State에 저장되는 정보

- 사용자 질문 (user_query)
- 검색 결과 (retrieved_docs)
- 답변 초안 (draft_answer)
- 근거 검증 상태 (grounding_status)
- 오류 목록 (errors)
- 실행 기록 (trace)



State는 단순한 기억 공간이 아닙니다. RAG Agent가 검색·답변 생성·근거 검증·Trace 기록을 이어가기 위한 실행 문맥입니다.

graph_state.py: State 구조 확인 파일의 역할

파일 역할

src/day2/graph_state.py는 RAG Agent가 사용할 State 구조를 보여주는 파일입니다.

01

State 구조 확인

사용자 질문, 알람 코드, 설비 ID, 검색 결과, 답변 초안, 최종 답변, Trace를 어떤 형태로 저장하는지 확인

02

State 변화 실습

검색 전과 검색 후 State가 어떻게 달라지는지 확인

03

Node 간 전달 구조 이해

다음 장에서 이 State가 여러 Node 사이를 이동하며 RAG Graph 흐름을 만드는 방식 확인

설계 관점

graph_state.py는 RAG Agent v1에서 검색 결과와 답변 생성 상태가 어떤 구조로 저장되고 전달되는지 확인하는 실습 파일입니다.



이 State 구조는 3일차 Multi-Agent 구조에서 Agent 간 Handoff State로 확장됩니다.

주요 State 구조 설계

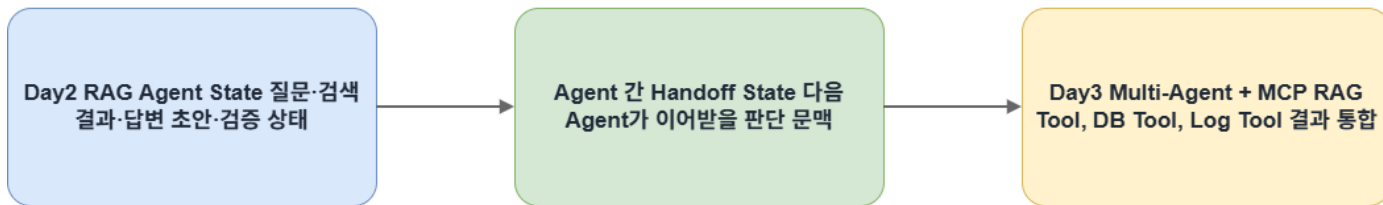
State 항목	의미	설계 관점
user_query	사용자 질문	전체 실행 흐름의 원본 입력
rewritten_query	재작성 질문	검색 실패 또는 근거 부족 시 재검색 조건
equipment_id	설비 ID	3일차 DB/Log Tool 호출 조건으로 확장 가능
alarm_code	알람 코드	RAG 검색과 알람 이력 조회 조건
retrieved_docs	검색된 문서 근거	답변 생성과 근거 검증의 핵심 입력
draft_answer	답변 초안	검증 전 임시 응답
final_answer	최종 답변	근거 검증 후 사용자에게 제공되는 응답
grounding_status	근거 사용 상태	답변이 검색 근거를 반영했는지 확인
needs_rewrite	재검색 필요 여부	조건부 분기 판단 기준
retry_count	재시도 횟수	무한 반복 방지 기준
errors	오류 목록	fallback 또는 안전 응답 판단 기준
trace	실행 기록	품질 평가와 디버깅 근거



State에는 질문부터 검색 결과, 답변, 오류, 실행 기록까지 저장되며, 이 구조가 Node 간 정보 전달의 기준이 됩니다.

State의 확장 방향

Day2 내부 실행 문맥은 Day3 Agent 간 Handoff State로 확장됩니다.



발표 핵심: State는 한 파일 안에서만 쓰는 저장 구조가 아니라, Multi-Agent 구조에서 도구 호출 결과와 판단 근거를 넘기는 연결 구조로 확장됩니다.

실행: graph_state.py

실행 위치 및 명령어

명령어는 manufacturing_agent_project 폴더의 루트 위치(src 폴더가 보이는 위치)에서 실행합니다.

```
uv run python src/day2/graph_state.py
```

실행 결과

- RAG Agent가 사용할 State 구조와 예시 변화 확인
- 검색 전 State와 검색 후 State 비교
- outputs/day2 폴더에서 결과 파일 확인

이 실행의 설계적 의미

RAG Agent가 검색 결과, 답변 초안, 근거 검증 상태를 어떤 State 구조로 관리하는지 확인하는 단계입니다.

- retrieved_docs에 검색 결과가 어떻게 저장되는지
- grounding_status가 어떻게 기록되는지
- trace에 실행 흐름이 어떻게 남는지

 이 구조는 이후 Node 실행 순서와 조건부 분기 설계의 기반이 됩니다.

state_demo_result.md

outputs/day2/state_demo_result.md는 State 구조와 변화 예시를 정리한 Markdown 산출물입니다. 검색 전 State와 검색 후 State의 차이를 확인할 수 있습니다.

retrieved_docs

Agent가 찾아온 문서 근거가 저장됩니다. 검색 전에는 비어 있고, 검색 후에는 Top-3 근거 후보가 채워집니다.

draft_answer

검색 결과를 바탕으로 생성된 답변 초안이 저장됩니다. 근거 검증 전 임시 응답입니다.

grounding_status

답변이 검색 근거를 충분히 반영했는지 확인하는 상태가 기록됩니다.

trace

Agent가 어떤 단계를 거쳤는지 실행 흐름이 저장됩니다. 4일차 Trace 기반 품질 평가의 기반이 됩니다.

- ✔ state_demo_result.md는 검색 전후 State 변화와 Trace 기록을 확인하여, 이후 Graph 실행에서 어떤 정보가 다음 Node로 전달되는지 점검하는 자료입니다.

검색 전 State와 검색 후 State 비교

구분	검색 전 State	검색 후 State	의미
user_query	사용자 질문 저장	그대로 유지	전체 실행 흐름의 원본 입력 유지
retrieved_docs	비어 있음	검색된 문서 목록 저장	Agent가 답변 근거 후보를 확보했다는 의미
draft_answer	없음	답변 초안 생성 가능	검색 결과를 바탕으로 답변 생성 준비
grounding_status	비어 있음	근거 상태 기록 가능	답변이 근거를 충분히 반영했는지 확인 기준
trace	초기 단계만 있음	검색 단계 기록 추가	Agent 실행 흐름이 추적 가능해짐

다음 단계 연결

검색 결과가 확보되면 이제 그 결과를 Agent가 다음 단계에서 사용할 수 있도록 State에 저장해야 합니다. 다음 장에서는 이 State가 여러 Node를 지나가며 어떻게 RAG Graph 흐름을 만드는지 살펴봅니다.

3일차 확장 방향

State는 검색 결과를 다음 Node로 넘기는 연결 구조입니다. 3일차 Multi-Agent 구조에서는 Agent 간 Handoff State로 확장되어 search_manual MCP Tool의 입출력 구조와 연결됩니다.

 State는 검색 결과를 다음 Node로 넘기는 연결 구조이며, 이후 3일차 Multi-Agent 구조에서는 Agent 간 Handoff State로 확장됩니다.

2일차 핵심 정리

2일차는 제조 기술 문서를 RAG 검색 대상으로 만들고, 검색 결과를 State와 Node 흐름에 연결하여 근거 기반 답변을 생성하는 구조를 이해하는 시간입니다.

Chunk + Metadata 설계

제조 기술 문서를 검색 가능한 지식 단위로 분할하고, metadata 기반 검색·필터링 구조 설계

Top-3 근거 후보 확보

rag_search.py로 관련 문서 chunk를 검색하고, 검색 품질을 rag_test_results.md에서 검토

State·Node 연결 구조

검색 결과를 State에 저장하고, Node 실행 순서와 grounding 검증 흐름으로 연결

3일차·4일차 확장

이 구조는 3일차 search_manual MCP Tool과 4일차 Trace 기반 품질 평가로 확장됩니다